Classroom follow-along

Work with a partner. You will write one useful instruction, change one request in a prepared program, and review an unsafe answer. The longer AI Coding Lab is optional practice after class.

Before class: get one local run working

Install Python 3.11 and one editor. Antigravity is optional; an existing editor works for every manual task. Extract Campus Canteen Student Kit.zip and open its demo folder. Follow demo/README.md for setup. Do this before class; package installation needs internet.

On Windows, from demo:

```
py -3.11 -m venv .venv
.venv\Scripts\python -m pip install -r requirements.txt
.venv\Scripts\python 02_react_agent.py --offline
```

On macOS/Linux, from demo:

```
python3.11 -m venv .venv
.venv/bin/python -m pip install -r requirements.txt
.venv/bin/python 02_react_agent.py --offline
```

If .venv already works, use it rather than creating it again. Python 3.11 must already be installed. No activation or execution-policy change is required. The Windows path was tested; macOS/Linux commands are equivalent guidance, not an executed check on those platforms.

Expected normal result: the Rs 60 vegetarian sandwich qualifies under Rs 80; the mock canteen is open and closes at 16:00. --offline uses a rule-based stand-in, not a real AI. If your device is blocked, pair up or open demo/saved_runs/02_agent_lunch_and_hours_OFFLINE.txt.

Optional: Google AI Studio live-model setup

1. Open Google AI Studio and sign in with an eligible account. AI Studio requires age 18+; use the offline exercises if access is unavailable.

2. Open API Keys. A new account may already have a default project/key. Otherwise use Create API key in the AI Studio interface and select/create a project there. This lesson does not require Google Cloud Console provisioning.

3. Check the project's free-tier status and available model/quota. The kit defaults to gemini-3.6-flash. Use a supported model with tool calling if account availability differs. Free-tier quota is limited; it is not a guaranteed cash balance or unlimited calls. No paid upgrade is part of this workshop.

4. Copy .env.example to .env only if you do not already have a configured file. Edit it privately in your editor. Set GOOGLE_API_KEY to your own key, without sharing it. Keep the GEMINI_MODEL entry. Never put a key in AGENTS.md, source code, chat screenshots or a shared submission.

5. From demo run the setup check, then the actual agent. These commands make real API calls:

```
.venv\Scripts\python check_setup.py
.venv\Scripts\python 02_react_agent.py
```

On macOS/Linux use .venv/bin/python. A text-call setup check alone does not prove agent tool calling works. If access, quota or a model blocks you, continue with --offline; do not buy credits just to finish the exercise. Your instructor must separately test the live presentation route. Signing into Antigravity does not automatically configure this Python project's key.

1. Give files a purpose

Open README.md, docs/PROJECT.md, AGENTS.md and data/menu.json. Say who reads each file. Current stock is data; instructions tell a coding assistant how to work; Python implements the checks. The ordinary Gemini API call does not automatically read these instruction files.

Create a scratch note with one Markdown title, a Requirements heading and two bullets. Preview it in your editor. Example:

```
# Canteen change

## Requirements

- Use Rs 30 as the default lunch budget.
- Keep the menu and filtering rules unchanged.
```

2. Write useful instructions

Read the supplied AGENTS.md. Write five short instruction lines on paper or in a scratch note; then add your best line to AGENTS.md without removing the existing guidance. For example:

```
- Read docs/PROJECT.md before editing.
- Make one focused change.
- Keep menu records unchanged.
- Check the final answer, including failures.
- Report what you ran and what remains untested.
```

Ask how someone would check each instruction. "Be brilliant" does not describe a checkable action. In a coding assistant, inspect the loaded project rules for your version. Current Antigravity supports workspace AGENTS.md and GEMINI.md; other clients have their own conventions. We practise one file here. Instructions guide the coder; runtime code enforces application rules.

3. Make one small change

From demo, copy 02_react_agent.py to practice_agent.py in the same folder. Use your editor's copy/save-as function. Keep the original example for comparison. Run the copy once with --offline before editing.

Find the line beginning QUESTION =. Change only its Rs 80 to Rs 30. Leave MENU_ONLY_QUESTION, tools and data unchanged. You may edit manually or ask your coding assistant:

Inspect practice_agent.py. Change only the default QUESTION budget from Rs 80 to Rs 30. Keep the tools, menu records and other request unchanged. Show the diff and explain the expected result. Do not make a live API call.

Inspect the proposed change before running it. From demo on Windows:

```
.venv\Scripts\python practice_agent.py --offline
```

On macOS/Linux use .venv/bin/python. Expected: no vegetarian food fits Rs 30. The sandwich costs Rs 60; poha costs Rs 35 and is unavailable. The hours answer is unchanged. A model may phrase the answer differently; check the facts.

Show your partner the changed line and the output. Explain why changing the budget should not change stock. If you finish early, try --scenario sold_out or --scenario menu_timeout and explain the difference between no matches and an unavailable source. For a clearer input-only comparison, run original demo 02 with --question "veg lunch under Rs 30".

4. Review the answer that reaches the user

Open Review Exercise.md or its one-page PDF. Inspect this unsafe pattern:

```
check(reply["recommended_dishes"])
return reply["message"]
```

Which value remains unchecked? Write one input where the dish list is valid but the message invents a wrap or false hours. State what the final answer should contain. Then watch the instructor run corrected demo 04. If practising alone:

```
.venv\Scripts\python 04_graph_with_agent.py --offline --inject-mistake free_text
```

The untrusted MODEL PROPOSAL contains the simulated mistake. ANSWER uses checked records. Explain why another instruction to "never hallucinate" would not repair an unchecked return value.

5. Show three things before leaving

1. One useful instruction and what a reviewer can check.

2. Your one-line budget change and the observed answer.

3. One review finding and the behavior a regression should check.

Use the full AI Coding Lab, worked exercises and graph implementations later. You do not need to learn every framework today.

Official setup references

- Gemini API keys
- Free-tier and billing guidance
- AI Studio access requirements
- Antigravity rules